

The Complete Guide to Databases

Choosing a store, the engine underneath it, indexes, data models, isolation levels and the operational failure modes — everything an HLD interview can ask about the layer your design actually depends on.

Pick a store on evidence

Access pattern first, brand second. Nine store families and what each one charges you.

B-tree vs LSM, properly

Pages, WAL, memtables, compaction, write and read amplification — and which wins when.

Indexes and models that scale

Leftmost prefix, covering indexes, keyset pagination, single-table design, ID choices.

Isolation and the real bugs

Lost update, write skew, MVCC, optimistic locking, deadlocks, replica lag, migrations.

What's inside

Read it front to back once. After that, Part 2 (engines), Part 5 (isolation) and Part 7 (interview answers) are the pages that earn the most points per minute of revision.

1 Choosing a datastore

Access patterns before products, the nine store families, SQL vs NoSQL without the marketing, and the real cost of a second store.

2 Storage engines under the hood

Pages and the write-ahead log, B-tree vs LSM tree, amplification, row vs columnar, what an fsync actually costs.

3 Indexing and query performance

What an index is, composite indexes and the leftmost-prefix rule, covering indexes, selectivity, the write cost, keyset pagination.

4 Data modelling for scale

Query-first modelling, denormalisation, single-table design, ID schemes, TTL and time-series tables, schema evolution.

5 Transactions and isolation

ACID honestly, the four isolation levels and the anomalies each one still allows, MVCC, write skew, optimistic vs pessimistic locking.

6 Running it in production

Connection pools, read replicas and lag, failover with RPO/RTO, online schema change, bloat, table partitioning.

7 The interview itself

How to answer a data-layer question, 14 asked-in-real-interviews Q&A, and the red flags to avoid.



One-page cheat sheet

The night-before page: the decision table, the numbers, the isolation grid, databases in eight facts.

Part of the HLD concept series. Caching, Redis, sharding, replication and the messaging layer are separate volumes — this one stops where the shard key begins.

How to use this guide

Understand

Every section opens in plain English before any jargon. If a sentence needs a diagram, there is one.

Say out loud

The grey “interview line” boxes are the exact sentences that earn the point. Practise saying them.

Defend

Every choice lists its cost. Interviewers score the trade-off, not the choice.

A NOTE ON SQL

The SQL in this volume is there because the query *is* the argument — an index that does or does not get used, a lock that is or is not taken. You are never asked to write production SQL on a whiteboard, but you are absolutely asked why a query is slow, and that answer is a sentence about pages, indexes and locks.

THE ONE THING TO INTERNALISE

A database is a set of trade-offs frozen into a product: how it writes to disk, what it locks, what it replicates and what it refuses to do. **Every data-layer question in an interview is asking whether you can name the trade-off you just accepted** — not whether you can name a product.

Choosing a datastore

PART 1

"Which database would you use?" is never really a question about databases. It is a question about your access patterns, and whether you derived the store from them or guessed.

1.1 Start from the access pattern, always

Before naming anything, say out loud what the workload does. Four numbers and two sentences are enough, and they eliminate most of the menu by themselves:

Read:write ratio

100:1 says cache and replicas. 1:1 says the write path is the design.

Access shape

Point lookups by key? Ranges by time? Multi-entity joins? Full-text? Aggregates?

Size and growth

Rows/day × bytes/row × retention. Say whether it fits one node — most designs do.

Consistency need

Per operation, not per system: money is strict, a like count is eventual.

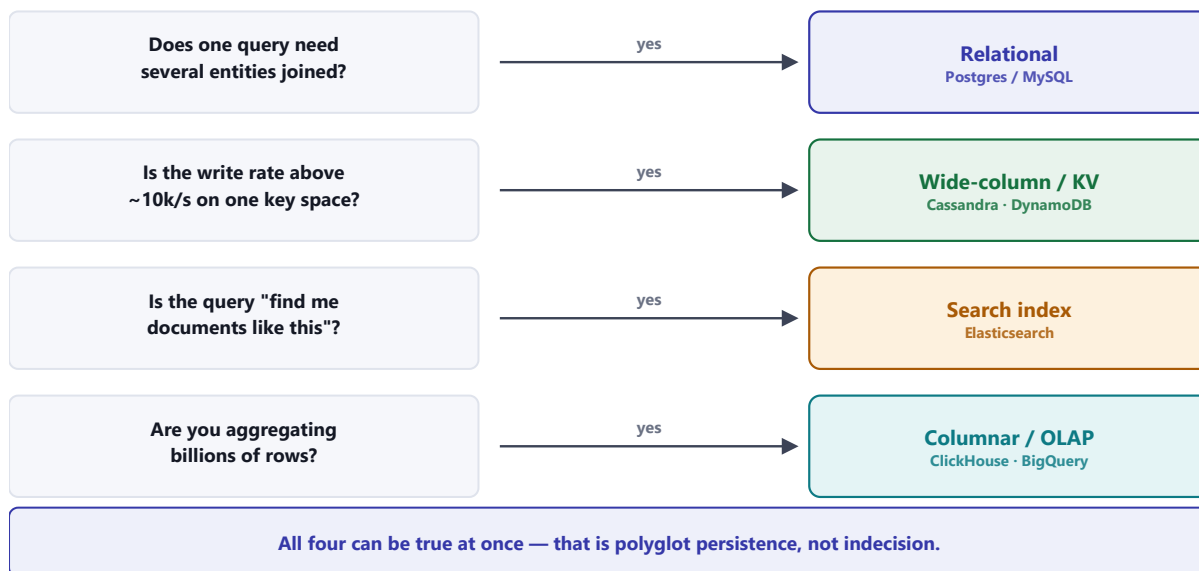


Figure 1.1 — The four questions that pick the store. Ask them in this order and the answer stops being a matter of taste.

1.2 The nine families, and what each one charges you

Products change; families do not. Name the family, then one product in it, then the bill it hands you.

Family	Reach for it when	What it charges you
Relational Postgres · MySQL	Joins, transactions across rows, ad-hoc queries, anything money touches	One primary for writes (~5–10k/s); schema changes need care; sharding is manual
Wide-column Cassandra · Scylla	Huge write rates, time-series by partition key, linear scale-out, multi-region writes	No joins; you model per query; eventual consistency unless you pay quorum latency
Managed KV DynamoDB	Predictable point access at any scale with no servers to run	Item and partition limits, GSI cost, and the bill tracks your access pattern
Document MongoDB	One aggregate read per request, flexible fields, per-document atomicity	Cross-document transactions are possible but not the design's strength
In-memory Redis · Memcached	Sub-millisecond reads, counters, leaderboards, sessions, locks	RAM cost, and it is a dependency the moment your DB is sized for a hit ratio
Search Elasticsearch	Free-text, relevance ranking, facets, typo tolerance	Near-real-time only (refresh interval); a second copy of the data to keep in step
Columnar / OLAP ClickHouse · BigQuery	Aggregates over billions of rows, dashboards, funnels	Not for point updates; batch or streaming ingest, not per-request writes
Time-series Prometheus · Timescale	Metrics, retention windows, downsampling, time-bucketed queries	Cardinality is the enemy — every label combination is a new series
Object store S3 · GCS	Blobs: images, video, backups, data-lake files	No queries. Store the key in your database, never the bytes

INTERVIEW LINE

"Before I pick a store I want the read:write ratio and the access shape, because those two decide it. If reads are point lookups by a known key and writes are heavy and append-shaped, I want an LSM-backed wide-column store. If the same request needs three entities joined and a transaction across them, I want Postgres — and I would rather add a cache than give that up."

1.3 SQL vs NoSQL, without the marketing

The honest framing is not “relational versus not”. It is **which guarantees am I willing to give up to get horizontal write scale** — because that is the actual trade.

What relational still wins

Joins you did not anticipate, transactions across entities, unique constraints the database enforces for you, and an ecosystem of tooling. A single Postgres node with an index and a cache handles more than most candidates assume: tens of thousands of reads/s and thousands of writes/s.

What it costs you

Writes funnel through one primary, so past roughly 10k writes/s on one table you are into partitioning, sharding or a different family. Schema change on a large table needs an online-migration tool, not an **ALTER** in a deploy.

Three claims to correct if the interviewer offers them:

- **“NoSQL is schemaless.”** The schema moved into your application code, where nothing enforces it. You still have one — you just cannot see it.
- **“NoSQL is faster.”** Per operation it is often the same order of magnitude. What it buys is *scale-out* of writes and predictable latency at size.
- **“Postgres does not scale.”** It scales vertically a long way, and reads scale with replicas. What does not scale is a single write primary — say that instead.

1.4 Polyglot persistence and its real bill

Serious systems run more than one store: Postgres for orders, Redis for sessions, Elasticsearch for search, S3 for files, a warehouse for analytics. That is normal. What a senior answer adds is the cost of the second copy:

Two truths

Which store is authoritative? Everything else is a projection, and projections need rebuilding.

A sync path

CDC, an outbox, or a dual write you will regret. Name it and its lag.

Twice the operations

Backups, upgrades, on-call, capacity, access control — per store.

A consistency story

“Search may be 2 s behind the order table, and here is why that is acceptable.”

Storage engines under the hood

PART 2

Two designs cover almost every store you will name in an interview: the B-tree that updates a page in place, and the log-structured merge tree that only ever appends. Knowing which one you chose explains your write path, your latency spikes and your disk bill.

2.1 The two things every engine does first

Pages. Storage is read and written in fixed blocks — 8 KB in Postgres, 16 KB in InnoDB. A row lives inside a page; a “random read” means fetching one page. The buffer pool (or page cache) keeps hot pages in RAM, which is why the same query is 100× faster the second time and why RAM sizing is a database decision, not an ops afterthought.

The write-ahead log. Before a change touches the data files it is appended to a log and flushed. Crash recovery replays the log. This is why a durable write costs a sequential disk flush, why **fsync** is the unit of write cost, and why group commit (batching many transactions into one flush) is the single biggest write-throughput knob in every relational engine.

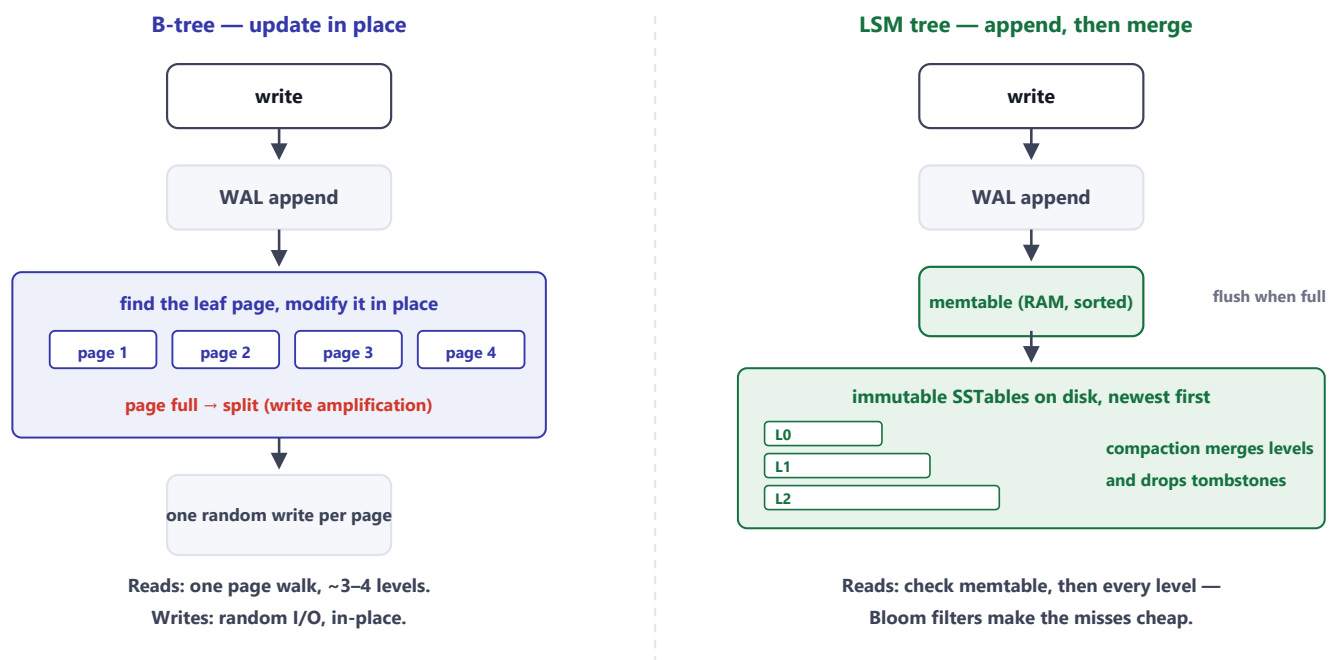


Figure 2.1 — The same write, two engines. The B-tree pays random I/O now; the LSM pays merge work later and read work on every lookup.

2.2 B-tree: sorted, balanced, updated in place

A B-tree keeps keys sorted in a shallow tree of pages — typically three or four levels for hundreds of millions of rows, so any lookup is a handful of page reads. A write finds the leaf page and modifies it. When a leaf is full it **splits**, which is where the write amplification and the occasional latency spike come from.

- **Reads are predictable.** Point lookups and range scans both walk the same structure, so *ORDER BY* on the index columns is free.
- **Writes are random I/O.** Two rows written together may live in pages far apart. On SSDs this is survivable; on spinning disks it was the whole story.
- **Fill factor matters.** Pages packed to 100% split on the next insert; leaving headroom trades space for fewer splits.
- **Monotonic keys are a mixed blessing.** Appending to the right edge avoids random I/O, but in a sharded world it puts every insert on one shard (Part 4.4).

2.3 LSM tree: append now, sort later

Writes land in an in-memory sorted structure (the **memtable**) plus the log. When it fills, it is flushed to an immutable **SSTable** on disk. Background **compaction** merges SSTables, drops overwritten values and removes **tombstones** — the markers that record a delete.

Property	What it means	Consequence you should name
Write amplification	Each byte is rewritten several times as it moves down the levels	Disk writes are 10–30× the logical write rate; SSD wear is real
Read amplification	A lookup may touch the memtable and one file per level	Bloom filters per SSTable make the misses ~free — say this
Space amplification	Old versions live until compaction reclaims them	Size-tiered compaction can transiently double disk; levelled trades write I/O for space
Tombstones	Deletes are writes, and they linger until compaction	Delete-heavy workloads read <i>slower</i> , the classic Cassandra trap

2.4 Which one wins, in one sentence

THE ONE-LINER THAT LANDS

“B-trees give me fast, predictable reads and in-place updates; LSM trees give me sequential writes and cheap ingest, and they claw the read cost back with Bloom filters and compaction. Write-heavy and append-shaped goes LSM; read-heavy with ranges and transactions goes B-tree.”

Where the engines actually live: B-tree in Postgres, MySQL/InnoDB, Oracle and most index structures; LSM in Cassandra, ScyllaDB, RocksDB (and therefore in TiKV, CockroachDB storage, Kafka Streams state stores), LevelDB, HBase and DynamoDB’s internals. MyRocks and Postgres extensions blur the line, which is a good aside if pushed.

2.5 Row store vs column store

Row-oriented (OLTP)

All columns of a row sit together, so “give me this order” is one page read. Aggregating one column across a billion rows means reading every row.

Column-oriented (OLAP)

Each column is stored and compressed separately, so **SUM(amount)** over a billion rows reads one column and nothing else — often 10–100× less I/O, with compression ratios that row stores cannot reach.

This is why analytics does not belong on your OLTP primary: it is not just load, it is the wrong physical layout. The standard answer is CDC or a nightly load into a columnar store, and the interview point is naming the freshness you accepted (“dashboards are five minutes behind”).

2.6 What durability costs, numerically

fsync on NVMe

~0.1–1 ms. It is the floor under every committed write.

Group commit

Batches N transactions into one flush — throughput scales, latency stays.

Sync replication

Adds a network round trip (0.5 ms local, 80 ms cross-region) to every commit.

Async replication

Commit returns before the replica has it: fast, and it can lose the tail on failover.

Indexing and query performance

PART 3

Most “the database is slow” stories are one missing or unusable index. This part is the vocabulary to diagnose that out loud: what an index costs, when the planner can use it, and when it silently cannot.

3.1 What an index actually is

A second, sorted copy of some columns plus a pointer to the row. It turns a full scan ($O(N)$ pages) into a tree walk (three or four page reads). Everything else about indexing follows from that one sentence: it is a copy, so writes maintain it; it is sorted, so only prefixes of that sort order can be searched.

3.2 Composite indexes and the leftmost-prefix rule

An index on (A, B, C) is sorted by A, then B within A, then C. You can seek on A, on A+B, or on A+B+C — never on B alone, because B is only sorted *inside* a value of A. Getting this right is the difference between a 2 ms query and a table scan.

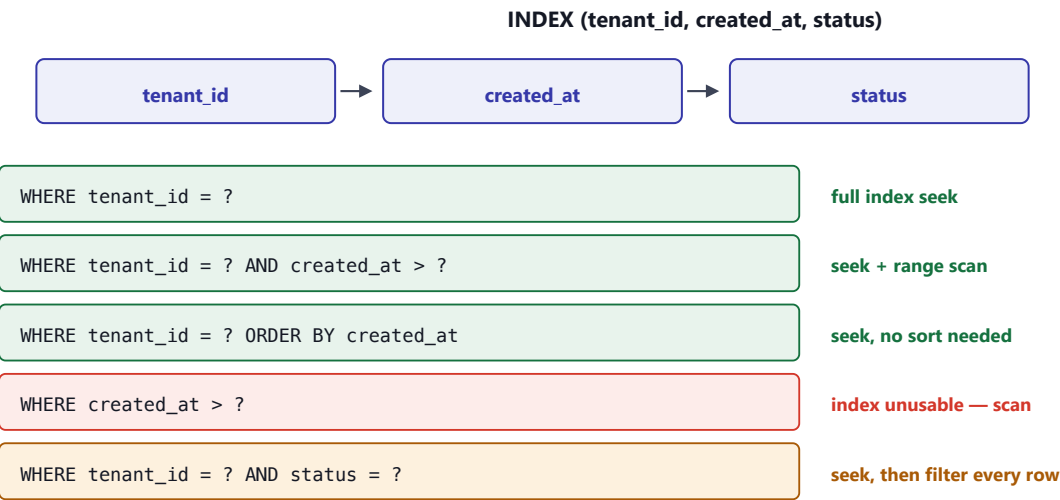


Figure 3.1 — One index, five queries. The rule is mechanical: equality columns first, then the range or sort column.

Ordering rule for a composite index: **equality predicates first, then the range or ORDER BY column, then anything you only want to avoid a row fetch for.** A range column placed before an equality column ends the useful part of the seek right there.

3.3 Covering indexes, selectivity, and why the planner ignores you

Covering / index-only scan

If the index carries every column the query needs, the row is never fetched. This is the cheapest big win in a read-heavy table.

Selectivity

How much of the table a predicate eliminates. An index on a boolean "is_active", 90% true, is nearly useless on its own.

Cardinality

Distinct values. High cardinality (user_id) indexes well; low cardinality (status) belongs later in a composite.

Statistics

The planner uses sampled histograms. Stale statistics after a bulk load is a real and common cause of a sudden plan change.

Reasons an index exists and still is not used — worth reciting, because interviewers ask "the index is there, why is it slow?":

- A function or cast wraps the column: **WHERE lower(email) = ?** cannot use an index on **email** — you need an expression index on **lower(email)**.
- The predicate is a leading wildcard (**LIKE '%foo'**): no sorted structure can help; that is what a search index or a trigram index is for.
- The predicate matches most of the table, so a sequential scan is genuinely cheaper than random row fetches. The planner is right and your query needs rethinking.
- Type mismatch between the column and the parameter, so the comparison is not index-sargable.
- **OR** across different columns, which often needs two indexes and a bitmap combination rather than one.

3.4 The write cost nobody mentions

Every index multiplies write work: one row insert with five indexes is six structures to maintain, six sets of pages to dirty, and six things to replicate. On a write-heavy table, deleting an unused index is a performance *improvement*. Say it — it shows you have run one of these systems, not just queried it.

3.5 Pagination: the offset trap

OFFSET — degrades with depth

```
SELECT * FROM posts
ORDER BY created_at DESC
LIMIT 20 OFFSET 100000;
```

The engine still produces and discards 100,000 rows. Page 5,000 is thousands of times more expensive than page 1 — and rows shift under the user as new posts arrive.

Keyset / cursor — flat cost

```
SELECT * FROM posts
WHERE (created_at, id) < (?, ?)
ORDER BY created_at DESC, id DESC
LIMIT 20;
```

Seeks straight to the cursor and reads 20 rows. Constant cost at any depth, stable under concurrent inserts. This is what every infinite feed uses.

Interview framing: "I would expose an opaque cursor, not a page number. Deep OFFSET is $O(\text{offset})$ and it double-shows rows when the list is changing underneath — a feed cannot use it."

3.6 Reading a query plan out loud

You will not be handed an **EXPLAIN ANALYZE** output on a whiteboard, but naming what you would look for is a strong signal. Four things, in order:



INTERVIEW LINE

"I would confirm it with the plan before adding anything: if the estimate and the actual row count disagree by orders of magnitude, the fix is statistics, not another index. If it is a sequential scan on a selective predicate, the fix is a composite index with the equality column first."

Data modelling for scale

PART 4

Relational modelling starts from the entities. Modelling for scale starts from the queries. Both are correct in their place, and being explicit about which one you are doing is what separates a considered design from a guess.

4.1 Query-first modelling

Write the three or four access patterns down before any table: “timeline for user X, newest first”, “order by id”, “all orders for a customer in the last 90 days”. In a relational design those become indexes. In a wide-column or KV design they become *tables* — one per access pattern, each denormalised, each written to on the same event.

4.2 Normalise, then denormalise deliberately

Normalised

One fact in one place, so updates are cheap and cannot disagree with themselves. Reads pay with joins.

Denormalised

The read is one lookup, so it scales and it shards. Writes pay: the same fact now lives in several rows, and keeping them in step is your job.

The senior sentence is the second half: **“I denormalise the read path and accept a fan-out write, and I make that write idempotent and replayable so a partial failure is recoverable.”** Denormalisation without saying how you repair drift is the junior version of the same answer.

4.3 One row per question: the counter example

A like count read a million times a second and written a thousand times a second is not a column on the post row — that row becomes a lock convoy. Split it: an append-only events table or a sharded counter for the writes, and a periodically materialised total for the reads. Same data, two shapes, because the two access patterns have nothing in common.

4.4 Choosing a primary key

Scheme	Shape	Use it when · watch out for
Auto-increment	Dense, monotonic, 8 bytes	Single-node relational. Leaks volume, needs the DB to mint it, and every insert hits the same page or shard
UUIDv4	Random, 16 bytes	Client can mint it offline. Random order destroys B-tree locality and bloats indexes — measurable on large tables
UUIDv7 / ULID	Time-ordered prefix + random	The modern default: client-generated, index-friendly, sortable by creation. Still 16 bytes
Snowflake	timestamp machine sequence, 8 bytes	When you need ordered, compact, coordination-free IDs at high rate — Twitter, Discord. Needs machine-ID assignment and clock discipline
Natural key	Email, ISBN, tenant slug	Only when it is genuinely immutable. People change emails; countries rename themselves

THE TRAP TO NAME

A monotonically increasing key is the best thing for a single B-tree (append at the right edge) and the worst thing for a sharded cluster (every write lands on the newest partition). If you shard on time or on an auto-increment id, you have built a hotspot — prefix with something high-cardinality, or hash the key.

4.5 Time-series and TTL tables

- **Partition by time** (daily or monthly) so retention is a **DROP PARTITION**, not a **DELETE** of a billion rows that bloats the table and floods replication.
- **Bucket the partition key** in wide-column stores: **(sensor_id, day)** keeps partitions bounded — an unbounded partition is the number-one Cassandra design mistake.
- **Let the store expire rows** where it can (DynamoDB TTL, Cassandra TTL, Timescale retention policies) instead of writing your own reaper.

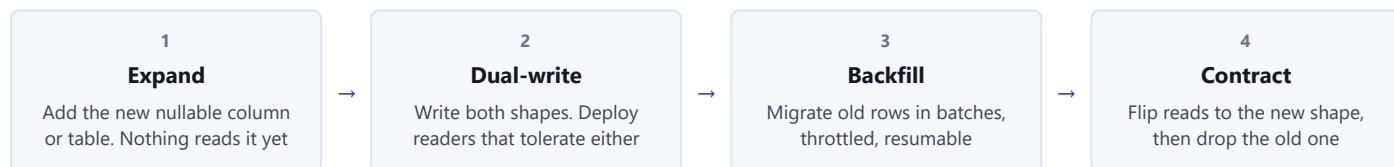
4.6 Single-table design, when the store is DynamoDB

In a KV/wide-column store you cannot join, so the join is done at write time by putting related items under one partition key with sortable prefixes. One query then returns a whole aggregate — user, their orders, their addresses — with a single round trip.

PK	SK	Item
USER#42	PROFILE	name, email, created_at
USER#42	ORDER#2026-08-01#1183	total, status
USER#42	ORDER#2026-08-14#1204	total, status
USER#42	ADDR#home	line1, city, postcode
ORDER#1204	ITEM#7	sku, qty, price

“Give me user 42 and their last 10 orders” is one query with **PK = USER#42 AND begins_with(SK, 'ORDER#')**, newest first. Access patterns you did not model need a **global secondary index** — which is a second copy, updated asynchronously, and therefore eventually consistent. Say that when you add one.

4.7 Schema evolution without downtime



This expand–contract sequence is the answer to “how do you change a schema on a live table?” — plus the tooling: **gh-ost** or **pt-online-schema-change** for MySQL, a concurrent index build and a **NOT VALID** constraint promoted later for Postgres. The failure to avoid is the migration that takes an **ACCESS EXCLUSIVE** lock on a hot table during peak traffic.

Transactions and isolation

PART 5 · THE DEEP-DIVE MAGNET

Nothing separates candidates faster than this part. Everyone can say “ACID”. Very few can name what READ COMMITTED still allows, and that is exactly the question interviewers reach for when they want to find your ceiling.

5.1 ACID, honestly

Atomicity

All of the statements or none. Implemented by the log, not by hope.

Consistency

Your invariants hold — the database only enforces the ones you declared as constraints.

Isolation

How much concurrent transactions can see of each other. This is the dial, and the default is not the top.

Durability

Committed means flushed. Note that “flushed on the primary” is not “present on a replica”.

5.2 The levels, and the anomaly each one still permits

Level	Still allows	Cost / where you meet it
READ UNCOMMITTED	Dirty reads — you see uncommitted data	Effectively unused; MySQL only
READ COMMITTED	Non-repeatable reads, phantoms, lost updates	The Postgres and Oracle default. Cheap, and the level most production bugs live at
REPEATABLE READ	Phantoms (classically) and write skew	The MySQL/InnoDB default; in Postgres this is snapshot isolation
SERIALIZABLE	Nothing — equivalent to some serial order	Retries and aborts under contention (SSI), or real locking. Correct, and you pay for it

The four anomalies in one line each: **dirty read** — seeing uncommitted data; **non-repeatable read** — the same row changes inside your transaction; **phantom** — a new row appears in a range you already queried; **lost update** — two read-modify-writes and one overwrites the other.

5.3 MVCC and snapshot isolation

Modern engines do not block readers. Each transaction sees a **snapshot**: rows carry version metadata, and a reader ignores versions committed after it started. Readers never block writers and writers never block readers — the property that made Postgres and InnoDB usable under load.

The costs are worth naming, because they are the operational failure modes of Part 6: old row versions must be reclaimed (**VACUUM**, purge threads), so a long-running transaction pins garbage and bloats tables; and a snapshot that is minutes old is a snapshot of minutes-old truth.

5.4 Write skew — the anomaly that survives snapshot isolation

Two on-call engineers, rule: at least one must stay on call

TX A — Alice goes off call

```
SELECT count(*) WHERE on_call  
→ 2, so it is safe to leave
```

```
UPDATE alice SET on_call = false  
one row, no conflict
```

```
COMMIT  
succeeds
```

TX B — Bob goes off call

```
SELECT count(*) WHERE on_call  
→ 2, so it is safe to leave
```

```
UPDATE bob SET on_call = false  
one row, no conflict
```

```
COMMIT  
succeeds
```

Both snapshots were valid. Nobody is on call. Snapshot isolation permits this; **SERIALIZABLE** or **SELECT ... FOR UPDATE** does not.

Figure 5.1 — Write skew: two transactions read overlapping data, write disjoint rows, and together break an invariant that each of them individually preserved.

THE FIXES, IN ORDER OF PREFERENCE

1. Put the invariant in the database: a unique index, a check constraint, an exclusion constraint. Nothing beats a constraint.
2. Take the conflict explicitly: **SELECT ... FOR UPDATE** on the rows the invariant spans, or a materialised “lock row” when the invariant spans a set.
3. Run that transaction at **SERIALIZABLE** and be ready to retry on abort.
4. Last resort: a distributed lock — which is an efficiency lock, not a correctness one.

5.5 Optimistic vs pessimistic, and the last item in stock

Optimistic — version check

```
-- read
SELECT qty, version FROM stock WHERE id = 9;

-- write only if nobody moved
UPDATE stock SET qty = qty - 1,
               version = version + 1
  WHERE id = 9 AND version = 41;
-- 0 rows updated → retry
```

No locks held while the user thinks. Best when conflicts are rare — and the retry path must exist, or you have just built a silent data-loss bug.

Pessimistic — lock the row

```
BEGIN;
SELECT qty FROM stock
  WHERE id = 9
  FOR UPDATE;  -- others wait

UPDATE stock SET qty = qty - 1
  WHERE id = 9;
COMMIT;
```

Correct under heavy contention and simple to reason about. Costs held locks, so it needs short transactions, a lock timeout, and an eye on deadlocks.

Best answer of all for a decrement: let the database do it in one statement — **UPDATE stock SET qty = qty - 1 WHERE id = 9 AND qty > 0** — and treat “0 rows affected” as out of stock. One statement, no read-modify-write, no lost update, and the invariant is enforced by the predicate.

5.6 Deadlocks, in one paragraph

Two transactions hold what the other needs. The engine detects the cycle and kills one, so your application must retry — that is not an error condition, it is normal operation under contention. You reduce them by touching rows in a consistent order, keeping transactions short, avoiding user think-time inside a transaction, and taking the strongest lock you will need up front rather than upgrading later.

Running it in production

PART 6

The failure modes that actually page you are rarely exotic. They are connection pools, replica lag, a migration at the wrong time, and a table that grew a hundred times without anyone re-reading its access pattern.

6.1 Connection pooling and the cliff

Every Postgres connection is a process with its own memory; every MySQL connection a thread. A few hundred is healthy, a few thousand is a queue. Serverless functions and large fleets hit that ceiling long before they hit CPU, which is why a pooler (**pgbouncer**, RDS Proxy, ProxySQL) sits in front and multiplexes thousands of client connections onto tens of server ones.

Size it, do not guess

Little's law: $\text{pool} = \text{throughput} \times \text{latency}$. 500 qps $\times$ 4 ms is ~ 2 connections busy, plus headroom.

Transaction pooling

The mode that actually saves connections — and it forbids session state and prepared statements across queries.

Bound the queue

A pool with no timeout converts a slow database into a hung application. Fail fast and shed.

Per-service pools

One noisy service should not be able to consume every connection the primary has.

6.2 Read replicas and the lag you must design for

Replicas scale reads, not writes, and they are behind — milliseconds normally, seconds under load, minutes during a bulk import. That produces the single most common user-visible bug in a replicated system: the user writes, is routed to a replica, and their own change is missing.

- **Read-your-writes:** pin that user's reads to the primary for a few seconds, or carry the write's log position and require a replica at least that fresh.
- **Monotonic reads:** route a session to one replica (sticky) so time never appears to run backwards.
- **Monitor the lag, not the replica count.** Lag is a first-class SLI, and it is what your failover decision depends on.

6.3 Failover, backups, and the two numbers to quote

RPO — recovery point

How much data you accept losing. Async replication makes this “the unreplicated tail”; sync makes it zero and costs latency.

RTO — recovery time

How long until you are serving again. Automated failover means seconds to a minute; a restore from backup means hours.

PITR

Continuous log archiving lets you restore to a timestamp — the only real answer to “someone ran a bad UPDATE”.

Restore drills

A backup you have never restored is not a backup. Saying this costs nothing and reads as experience.

Failover is not free even when it works: connections drop, the new primary has a cold cache, and any client caching the old topology needs to re-resolve. And a promotion under async replication is a deliberate decision to lose the tail — the interview point is that *someone* must choose availability over that data, explicitly.

6.4 Table partitioning is not sharding

Partitioning (one node)

One logical table split into physical partitions by range or hash. The engine prunes partitions per query and retention becomes a DROP. Still one primary, still one machine’s write capacity.

Sharding (many nodes)

The data set split across independent nodes with a routing layer above. Adds write capacity, and takes away cross-shard joins and single-node transactions. That trade-off gets its own volume.

6.5 The four operational smells

Smell	What is really happening	Move
Table bloat	Long transactions or heavy updates leave dead versions VACUUM cannot reclaim	Kill long idle-in-transaction sessions; autovacuum tuning; pg_repack
Sudden plan flip	Statistics went stale after a bulk load, or a parameter sniffed badly	ANALYZE after loads; check estimate vs actual; pin the plan only as a last resort
Hot row	Every transaction updates the same counter or state row	Shard the counter, batch the updates, or move it to Redis with periodic materialisation
Queue in a table	Polling SELECT ... FOR UPDATE SKIP LOCKED at scale, plus dead tuples	Fine at low rate and honest about it; past that, a real broker

The interview itself

PART 7

Data-layer questions arrive in two shapes: “which database?” and “why is this slow / wrong?”. The first wants a derivation, the second wants a mechanism. Both want you to name the cost of your answer.

7.1 A four-step answer that always works



7.2 The five sentences that earn points

- “Different operations in this system deserve different consistency — the payment is strict, the view count is eventual.”
- “This fits on one Postgres primary with a cache and replicas; I would not shard until the write rate or the data size forces it.”
- “I would model this per access pattern, so the read is one lookup and the write is a fan-out I make idempotent.”
- “That is a lost update, so I need either one atomic statement, a version check, or a row lock — and here is which I would choose.”
- “The migration is expand, dual-write, backfill, contract, and every step is reversible.”

THE QUESTION BEHIND THE QUESTION

When an interviewer asks “SQL or NoSQL?”, they are testing whether you understand what you lose. Answer with the loss: “Cassandra gives me linear write scale and I give up joins, ad-hoc queries and multi-row transactions — so I model every read up front and accept that a new query means a new table.”

7.3 Fourteen questions with model answers

Q1 How would you choose between Postgres and Cassandra for this feature?

Derive it. If the request needs several entities together, needs a transaction across them, or the query set will keep changing, Postgres — and I would scale reads with replicas and a cache before considering anything else. If writes are tens of thousands per second, partitioned naturally by an entity id, and every read is a point or range lookup inside one partition, Cassandra.

Signal: volunteering the ceiling of the simpler option: “one primary gives me roughly 5–10k writes/s; below that, sharding is complexity I have not earned.”

Q2 B-tree or LSM — what is the difference and when does it matter?

B-trees update a page in place: predictable reads, random write I/O, page splits. LSM trees append to a memtable and flush immutable SSTables that compaction merges later: sequential writes, cheap ingest, and reads that may touch several levels, which Bloom filters make survivable. It matters when the workload is write-heavy — that is when you pick LSM and accept write amplification and delete-heavy read slowdowns from tombstones.

Q3 This query is slow and the index exists. What do you check?

Whether the index can actually be used. A function or cast around the column, a leading wildcard, a type mismatch, or a predicate on the second column of a composite index all make it unusable. Then whether it *should* be used: if the predicate matches most of the table, a sequential scan is cheaper and the planner is right. Then statistics — an estimate that is 100× off after a bulk load explains a sudden plan change.

Bonus: suggest a covering index so the query never fetches the row at all.

Q4 Design the indexes for “orders for a tenant in a date range, newest first”.

One composite index on **(tenant_id, created_at DESC)**: equality column first, then the range and sort column, so the seek lands on the tenant and reads the range in order with no sort step. If the list view needs status and total, I would include them so the scan is index-only. I would not add a separate index per column — that gives the planner three weak options instead of one good one.

Q5 Two users buy the last item at the same time. Walk me through it.

The bug is a lost update: both read `qty = 1`, both write `0`, both ship. Three fixes, best first. **One atomic statement:** `UPDATE stock SET qty = qty - 1 WHERE id = ? AND qty > 0` and treat zero rows affected as sold out. **Optimistic:** a version column and a retry loop. **Pessimistic:** `SELECT ... FOR UPDATE` and a short transaction. Which I pick depends on contention: for one hot SKU in a flash sale I would serialise per item rather than let thousands of retries thrash.

Q6 What does READ COMMITTED still allow?

Non-repeatable reads (the same row changes inside your transaction), phantoms (new rows appear in a range you already read) and lost updates from read-modify-write pairs. It is the Postgres default, so those anomalies are the normal state of most production systems — you handle them per operation with atomic statements, version checks, row locks or a stronger level on the transactions that need it.

Follow-up trap: “so use `SERIALIZABLE` everywhere?” — no. It converts contention into aborts, so it needs a retry path and it costs throughput. Use it where the invariant is worth it.

Q7 What is write skew and how is it different from a lost update?

A lost update is two transactions writing the *same* row. Write skew is two transactions reading an overlapping set, writing *different* rows, and together breaking an invariant over that set — the classic being two doctors both going off call because each saw the other still on. Snapshot isolation does not prevent it, because neither transaction touched a row the other wrote. Fix it with a constraint, an explicit **FOR UPDATE** over the set, or `SERIALIZABLE`.

Q8 A user updates their profile and immediately sees the old value. Why?

Replica lag: the write went to the primary, the read was routed to a replica that has not applied it. Fix with read-your-writes — pin that user’s reads to the primary for a few seconds after a write, or pass the write’s log position and require a replica at least that fresh. Sticky routing to one replica also gives monotonic reads so the value does not flip back and forth.

Q9 How do you add a NOT NULL column to a 500-million-row table with no downtime?

Expand, dual-write, backfill, contract. Add the column nullable with no default rewrite; deploy code that writes both and tolerates either; backfill in throttled, resumable batches; then flip reads and add the constraint — in Postgres as **NOT VALID** first and validated afterwards so the table is never exclusively locked, or with `gh-ost` on MySQL. The thing to avoid is a single DDL statement that takes an exclusive lock on a hot table at peak.

Q10 How do you paginate a feed of 10 million rows?

Keyset pagination on the sort key, not OFFSET. **WHERE (created_at, id) < (?, ?) ORDER BY created_at DESC, id DESC LIMIT 20**, with the cursor returned to the client as an opaque token. Constant cost at any depth, and stable when rows are being inserted — OFFSET is O(offset) and shows duplicates in a moving list.

Q11 What primary key would you use, and why does it matter?

UUIDv7 or a Snowflake id for anything distributed: client-generatable, time-ordered so B-tree inserts stay local, and sortable by creation. UUIDv4 is fine for low volume but its randomness hurts index locality on big tables. Auto-increment is fine on one node and bad in a sharded world, because a monotonic key sends every insert to the newest shard.

Q12 Your app runs out of database connections under load. What is happening?

Each connection is a process or thread, so a few thousand of them is a queue, not capacity. Usually it is a fleet that grew, a pool with no upper bound or timeout, or transactions held open across a slow external call. Put a pooler in front in transaction mode, size pools from throughput × latency rather than by hope, give checkout a timeout so the app sheds instead of hanging, and stop doing network I/O inside a transaction.

Q13 When would you deliberately keep two copies of the same data?

Whenever the two access patterns are physically incompatible: the transactional row store plus a search index, plus a columnar store for analytics. The rules I would state are which copy is authoritative, how the others are fed — CDC or an outbox, never a best-effort dual write — and what staleness the product accepts. The cost is operational: two systems to back up, upgrade and be paged for.

Q14 How do you delete a billion rows?

Not with one **DELETE** — that builds a huge transaction, bloats the table, floods replication and may lock. If the data is time-shaped, partition by time and **DROP PARTITION**, which is a metadata operation. Otherwise batch it: delete by key range in chunks of a few thousand with a pause, watching replica lag, and let the reclaim run afterwards. If the table is mostly going away, copy the survivors into a new table and swap.

7.4 Red flags interviewers listen for

Saying this	Says this about you
"I'd use MongoDB because it's schemaless"	The schema moved to your code, unenforced
Naming a product before naming the access pattern	Pattern-matching rather than designing
"Postgres doesn't scale"	Has never measured one; conflates read scale with write scale
Adding an index per column	Has not thought about composite order or write cost
OFFSET pagination on a feed	Has not run a deep page in production
"Transactions handle that" with no isolation level	Does not know what the default still allows
Sharding on an auto-increment id or a timestamp	Has built a write hotspot
Dual-writing to two stores in application code	No idea how the two diverge, and no repair path
"We'll add a read replica" for a write bottleneck	Replicas do not take writes
Analytics queries on the OLTP primary	Wrong physical layout, and one report can take the site down

7.5 Real systems worth name-dropping

Amazon Dynamo (2007)

Where consistent hashing, quorums, vector clocks and hinted handoff entered mainstream design. Cite it for leaderless replication and tunable consistency.

Google Spanner

External consistency at global scale using TrueTime and Paxos groups. The honest counterexample to "CAP means you cannot have both" — you pay in hardware and latency.

Facebook TAO / MyRocks

A graph-shaped cache over MySQL, and later an LSM engine under it. Cite it for "the access pattern chose the engine".

Uber's Postgres → MySQL write-up

A widely read account of write amplification, replication and connection handling deciding an engine choice. Good for showing you read post-mortems, not just docs.

One-page cheat sheet

The night-before page. Numbers first, because they end arguments.

The numbers

one primary ~ 5–10k writes/s · indexed read ~ 5–10 ms · fsync ~ 0.1–1 ms

RAM is ~100,000× faster than disk, a local round trip is 0.5 ms and a cross-region one is 80–100 ms. Those three facts explain caching, replicas and why you do not call across regions on the hot path.

The decisions

Question	Default answer
Which store?	Start relational; move only when a named pattern forces it
Engine?	Read-heavy with ranges → B-tree. Write-heavy and append-shaped → LSM
Index?	One composite: equality columns first, then the range/sort column
Primary key?	UUIDv7 or Snowflake — time-ordered, client-generatable
Isolation?	READ COMMITTED plus per-operation protection; SERIALIZABLE where the invariant is worth the retries
Decrement a counter?	One conditional UPDATE, not read-modify-write
Pagination?	Keyset with an opaque cursor. Never deep OFFSET
Schema change?	Expand → dual-write → backfill → contract
Analytics?	Off the primary: CDC into a columnar store, and state the lag

The isolation grid

Level	Dirty read	Non-repeatable	Phantom	Lost update / skew
READ UNCOMMITTED	possible	possible	possible	possible
READ COMMITTED	no	possible	possible	possible
REPEATABLE READ / snapshot	no	no	engine-dependent	write skew possible
SERIALIZABLE	no	no	no	no — pay in aborts

Databases in eight facts

Access pattern picks the store

Ratio, shape, size, consistency. Name those four and the product is nearly decided.

Writes go through one primary

Until you shard. Replicas add read capacity and lag, never write capacity.

An index is a sorted copy

So writes maintain it, and only prefixes of its sort order can be searched.

The default level is weak

READ COMMITTED allows lost updates. Protect the operation, not the whole system.

One statement beats read-modify-write

Conditional UPDATE, INSERT ... ON CONFLICT, or a constraint — let the engine be atomic.

Monotonic keys are a hotspot

Great for a local B-tree, terrible for a sharded cluster. Hash or prefix them.

Deletes are writes

Tombstones and dead versions cost reads until compaction or VACUUM catches up.

Migrations are four steps

Expand, dual-write, backfill, contract — each one reversible on its own.

IF YOU SAY NOTHING ELSE, SAY THIS

"I choose a store from the access pattern, not the brand: the read:write ratio, the query shape, the size and the consistency each operation actually needs. I start relational because it enforces invariants for me, I scale reads with a cache and replicas, and I only give up joins and single-node transactions when a named write-rate or data-size limit forces me to. Whatever I pick, I can tell you what it writes to disk, what it locks, and what it silently allows."

Sources & further reading — Martin Kleppmann, *Designing Data-Intensive Applications* (chapters 3, 5, 7 in particular); the PostgreSQL documentation on MVCC, isolation levels and index types; *Use The Index, Luke!* by Markus Winand for composite indexes and pagination; the Dynamo (SOSP 2007) and Spanner (OSDI 2012) papers; Facebook's TAO and MyRocks papers; RocksDB and Cassandra documentation on compaction strategies and tombstones; gh-ost and pt-online-schema-change docs; Brandur Leach's writing on Postgres operations; AWS documentation on DynamoDB single-table design and GSIs.